



# Classification and Logistic Regression



# Course Objectives

What is Classification Learning

Logistic Regression

How to validate a Classification Model?

# Learning Outcomes

At the end of the course, you will be able to

- Understand what is a classification learning
- Be familiar with the steps required to build a classification model
- Understand what is a logistic regression
- Be familiar with how to validate a classification model

# Machine Learning - Classification

## Object Recognition



Features – texture, weight, colour  
Label – apple / orange

Data  
Preparation

Feeding of  
data

Training the machine /  
Pattern Discovery

Object  
Recognition

# Machine Learning - Classification

Will I get grade A for my  
paper?

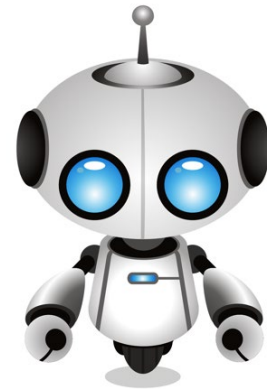


# The Outcome



Will I get grade  
A for my paper?

Based on your exam  
preparation time and  
participation rate, my  
machine says you got a 'B'



# Classification ML Model Building

*Problem Statement*

Classify a student to be an A or B grader

*Data Preparation*

Scan records, identify relevant attributes

*train*

**Model**

Exam Preparation Time =  $x_1$ ,  
Assignment Participation =  $x_2$ ,...

Classification outcome = "A" / "B"?

*Prediction*

# Classification

## Supervised Learning

| Student ID | Exam Preparation Time | Assignment Participation Rate | ... | Grade |
|------------|-----------------------|-------------------------------|-----|-------|
| A00011A    | 120                   | 80                            |     | A     |
| A00012B    | 140                   | 85                            |     | A     |
| A00013C    | 80                    | 20                            |     | B     |
| A00014D    | 85                    | 35                            |     | B     |
| ...        | ...                   | ...                           |     | ...   |

features

label



# Classification

## Supervised Learning

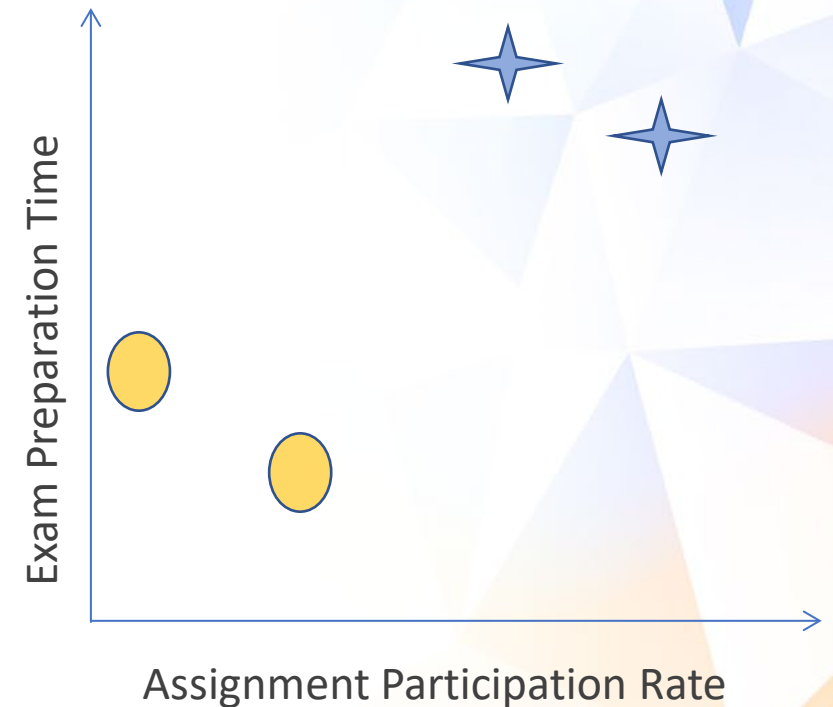
| Student ID | Exam Preparation Time | Assignment Participation Rate | ... | Grade |
|------------|-----------------------|-------------------------------|-----|-------|
| A00011A    | 120                   | 80                            |     | A     |
| A00012B    | 140                   | 85                            |     | A     |
| A00013C    | 80                    | 20                            |     | B     |
| A00014D    | 85                    | 35                            |     | B     |
| ...        | ...                   | ...                           |     | ...   |

features

label

★ = A(above average)

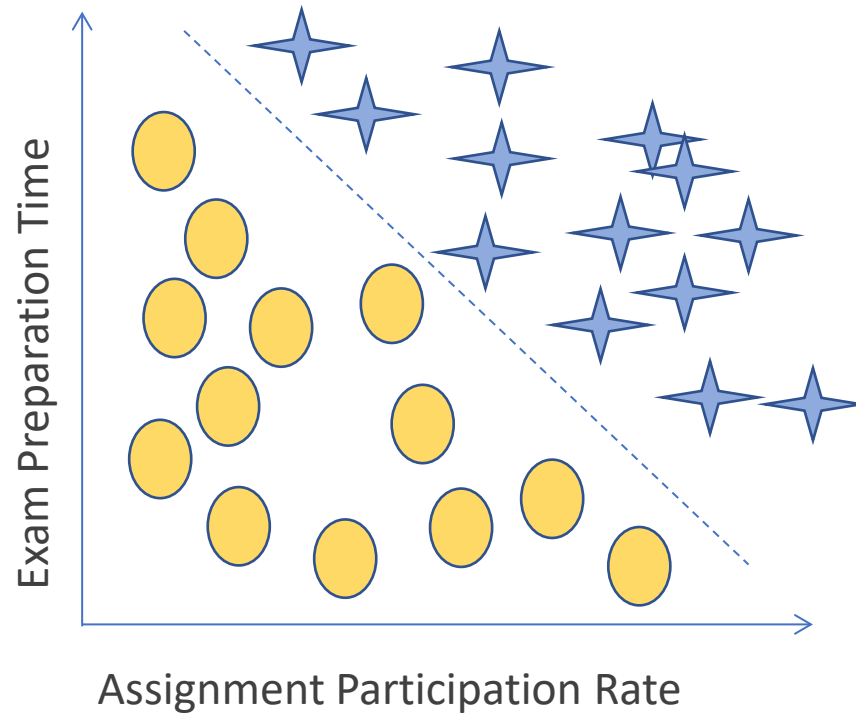
● = B(below average)



# Classification

★ = A(above average)

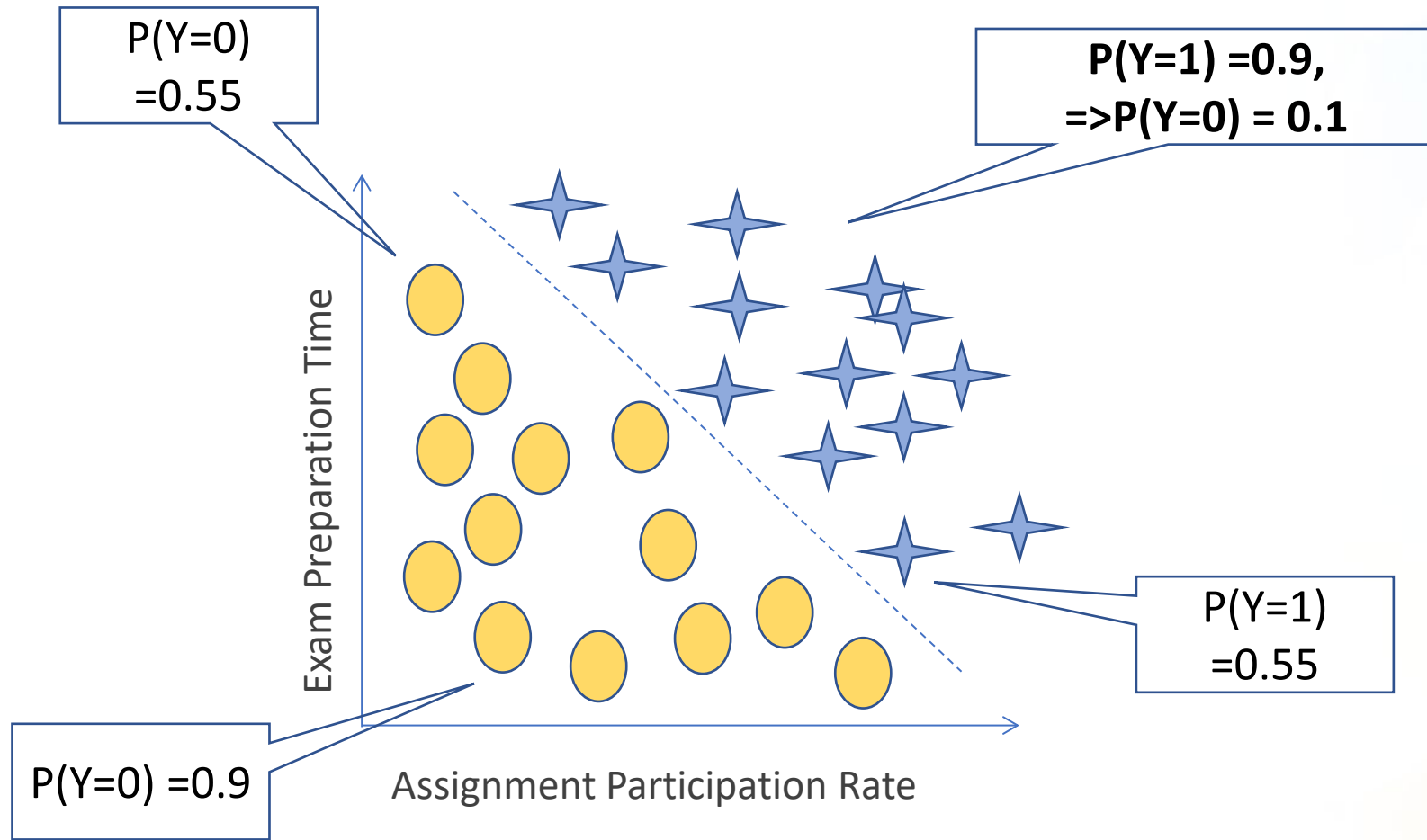
● = B(below average)



# Logistic Regression

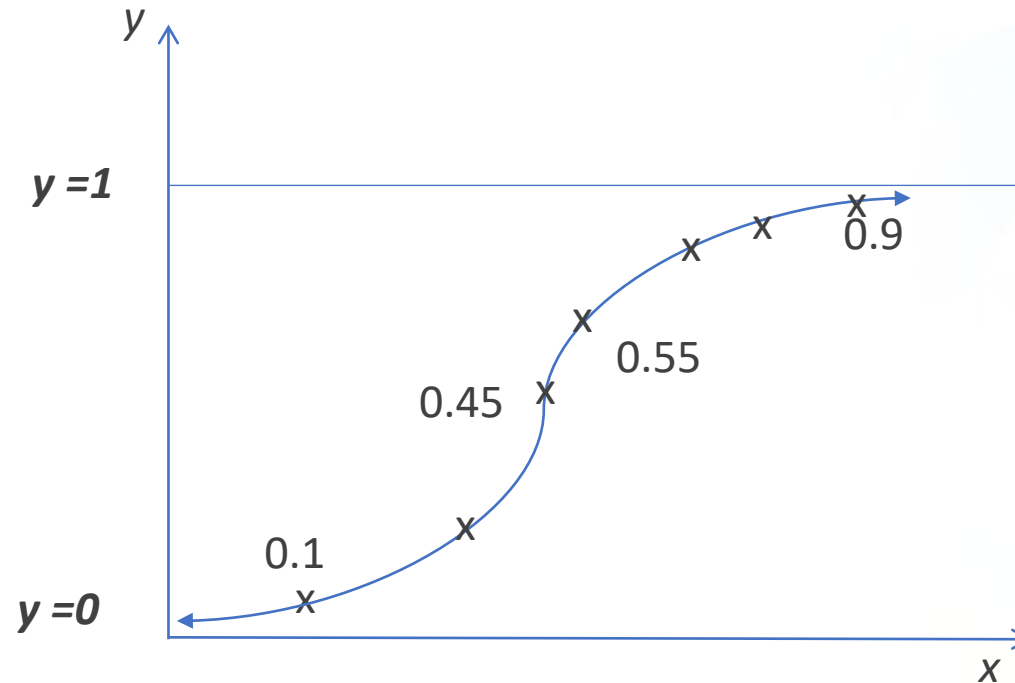
★ A(above average)  $\Rightarrow Y$  (outcome) = 1

● B(below average)  $\Rightarrow Y$  (outcome) = 0



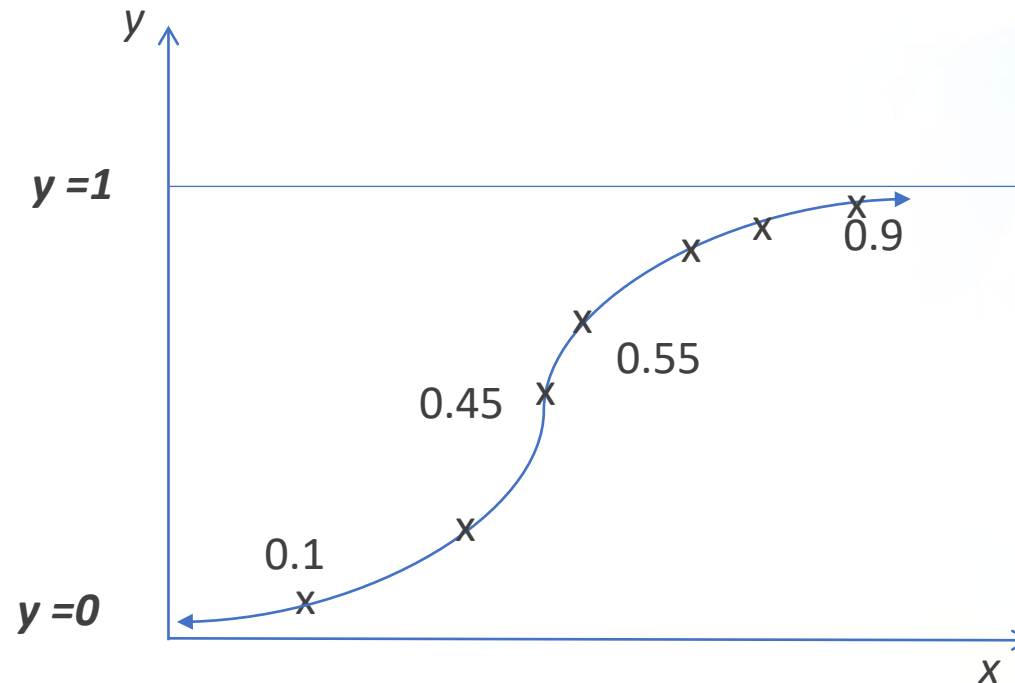
# Logistic Regression – How it works

*sigmoid function:*  $P(Y=1) = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + b_2 x_2)}}$



# Logistic Regression – How it works

*sigmoid function:*  $P(Y=1) = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + b_2 x_2)}}$



Classify the label as follows:

- 1 - probability  $p > 0.5$
- 0 - probability  $p < 0.5$

# Classification ML Model Building

*Problem Statement*

Classify a student to be an A or B grader

*Data Preparation*

Scan records, identify relevant attributes

*train*

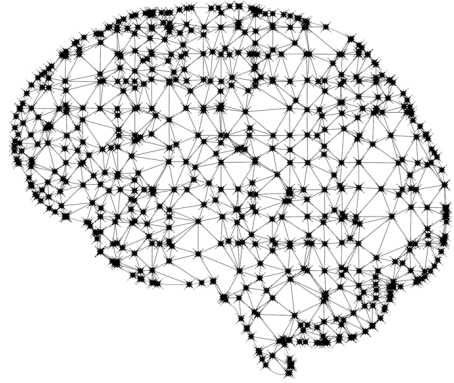
**Model**

Exam Preparation Time =  $x_1$ ,  
Assignment Participation =  $x_2$ ,...

Classification outcome = "A" / "B"?

*Prediction*

# Validation of the Model

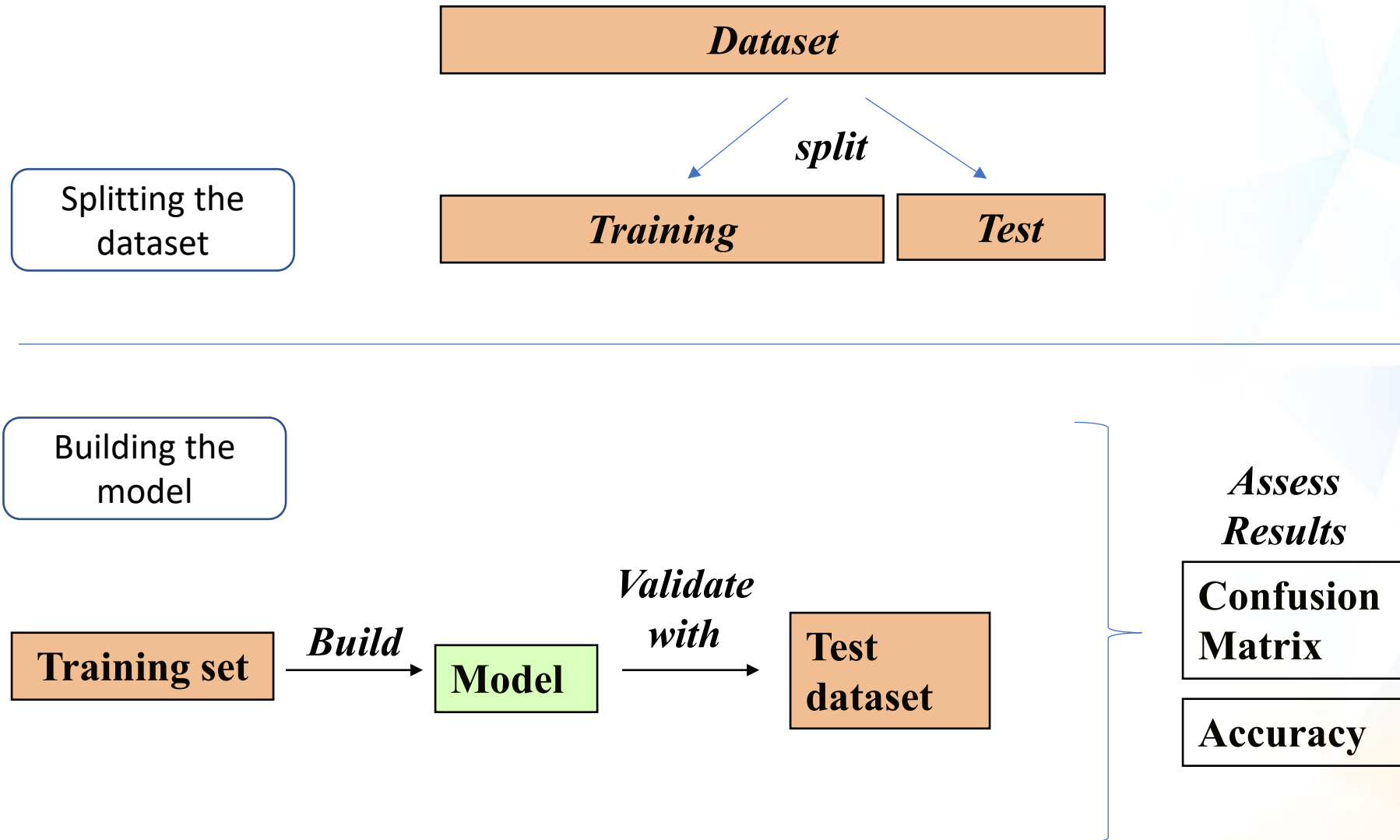


Trained ML model

How accurate is the prediction?



# Model Validation – Classification Learning





# Validating The Model

Prepare test dataset

Send selected feature to  
trained ML model for  
prediction

Compare actual label  
(in test dataset) with  
predicted label

| Exam<br>Preparation<br>Time | Assignment<br>Participation<br>Rate | ... | Grade |
|-----------------------------|-------------------------------------|-----|-------|
| 100                         | 35                                  |     | A     |
| 99                          | 35                                  |     | A     |
| 82                          | 40                                  |     | B     |
| 80                          | 30                                  |     | A     |
|                             |                                     |     | ..    |

relevant features

actual label

Trained ML model

Match?

Predicted  
outcome

| Grade |
|-------|
| A     |
| A     |
| B     |
| A     |
|       |

# Determine Accuracy

- How many correct and incorrect guesses?

|          | Predicted 0 | Predicted 1 |
|----------|-------------|-------------|
| Actual 0 | 73          | 3           |
| Actual 1 | 1           | 23          |

*Confusion Matrix*

$$\text{Accuracy score} = \frac{73+23}{73+3+1+23} = 96\%$$

# Confusion Matrix

|                     | Predicted Positive (0) | Predicted Negative(1) |
|---------------------|------------------------|-----------------------|
| Actual Positive (0) | True Positive          | False Negative        |
| Actual Negative (1) | False Positive         | True Negative         |

**True Positive:** These are cases in which the model correctly predicts the positive class.

- **Example:** If the task is to detect spam emails, a true positive would be an email that is correctly identified as spam.

**True Negative:** These are cases in which the model correctly predicts the negative class.

- **Example:** An email that is legitimately not spam (a regular email) is correctly identified by the system as not spam.

**False Positive:** These occur when the model incorrectly predicts the positive class.

- **Example:** In spam detection, a false positive would be a legitimate email that is incorrectly marked as spam.

**False Negative:** These occur when the model incorrectly predicts the negative class.

- **Example:** In spam detection, a false negative would be a spam email that is incorrectly marked as not spam.

# F1-Score

- Measure of a model's accuracy on a dataset.
- **Precision:** Number of **correct positive** results/Number of **all positive results predicted** by the classifier
- **Recall:** Number of correct positive results divided by the number of all **actual positive** instances
- F1-score ranges from 0 to 1, where 1 indicates **perfect precision and recall**, and 0 indicates the **worst performance**.

$$\text{Precision (Depend on Prediction)} = \frac{TP}{TP+FP}$$

$$\text{Recall (Depend on Truth)} = \frac{TP}{TP+FN}$$

$$\begin{aligned}\text{F1 Score} &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \times TP}{TP + 1/2(FP + FN)}\end{aligned}$$

# F1-Score – Example

**Scenario:** Consider a model that predicts whether emails are spam or not.

## Precision:

- **Predictions:** Suppose out of 100 emails predicted as spam, 70 are actually spam, and 30 are not (false positives).
- **Calculation:**  $\text{Precision} = 70 / (70 + 30)$
- **Interpretation: 70% of the emails predicted as spam are spam.**

## Recall:

- **Actual Positives:** Suppose there are 150 actual spam emails, and the model correctly identifies 70 of them, missing 80 (false negatives).
- **Calculation:**  $\text{Recall} = 70 / (70 + 80) = 70 / 150 \approx 0.47$
- **Interpretation: The model correctly identifies 47% of actual spam emails.**

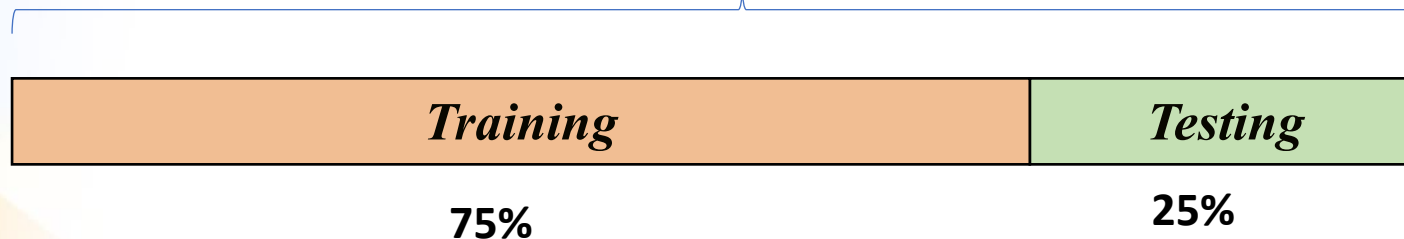
# Splitting Dataset

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

- 'Train\_test\_split' is to split dataset into the desired proportions of data for training and testing
- It returns the list containing train-test split of inputs to that function. In this case, 2 sets of inputs 'x' and 'y'.
- Default of test-size is 25% or you can specify any.
- E.g. 20% (test size)

Whole Dataset



```
>>> X
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> list(y)
[0, 1, 2, 3, 4]
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.33, random_state=42)
...
>>> X_train
array([[4, 5],
       [0, 1],
       [6, 7]])
>>> y_train
[2, 0, 3]
>>> X_test
array([[2, 3],
       [8, 9]])
>>> y_test
[1, 4]
```

# Difference between Linear and Logistic Regression

| Properties Comparison          | Linear Regression                                          | Logistic Regression                               |
|--------------------------------|------------------------------------------------------------|---------------------------------------------------|
| Type of Variables              | Numeric                                                    | Categorical                                       |
| Curve Type                     | Straight Line                                              | S Type Curve                                      |
| Relationship between DV and IV | Requires to have a relationship between them               | May not need to have a relationship between them  |
| Purpose                        | Use to predict the DV in case of a change of IV            | Calculate the probability of an event or category |
| Examples                       | Relationship between the duration of work hours and salary | Whether they are male or female                   |

# Classification ML Model Building

*Problem Statement*

Classify a student to be an A or B grader

*Data Preparation*

Scan records, identify relevant  
attributes

*train*

*Validation*

**Model**

Exam Preparation Time =  $x_1$ ,  
Assignment Participation =  $x_2$ ,...

Classification outcome = "A" / "B"?

*Prediction*



# What Have We Learnt?

- Classification Learning
- Logistic Regression
- Validation of Classification Learning

# Articles

- <https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f>
- [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1\\_score.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html)